MESA SCHOOL OF BUSINESS

Data Foundations · Pre-Read

Session 5: Urban Company

Big Mastery Question: Ops says 'deluxe services in Bangalore are the problem.' The answer is buried inside the booking codes — get it out.

Before you come to class

Read this once, slowly. This session is about text — messy, human-typed text, exactly like real startup data actually looks. You'll learn to slice codes apart, clean up sloppy typing, and stitch clean text back together. Every table below shows real column letters and row numbers, as always.

The company: Urban Company sends verified professionals — salon artists, cleaners, electricians, AC technicians — to your home for a booked service. Every booking gets a service_code that packs the city, category and tier into one short string, like BLR-SAL-DLX-02. Your job this session: pull the city, category and tier back OUT of that code, and clean up messy partner names along the way, so ops can finally see whether "deluxe in Bangalore" really is the problem.

0. The Dataset You'll Be Working With

Urban Company's bookings table has 9 columns (booking_id, booking_date, service_code, partner_id, partner_name, customer_pincode, price, rating, cancel_reason_text). Here's a 6-column, 6-row slice — deliberately messy, the way real data actually looks:

	A	C	E	G	H	I
1	booking_id	service_code	partner_name	price	rating	cancel_reason_text
2	U5001	BLR-SAL-DLX-02	priya sharma	1200	5	
3	U5002	HYD-CLN-STD-05	RAHUL VERMA	450	3	partner arrived late
4	U5003	BLR-ACR-REP-01	Fatima Khan	800	4	
5	U5004	DEL-PLM-STD-03	amit singh	350	2	customer not reachable
6	U5005	BLR-SAL-DLX-04	Priya Sharma	1250	5	
7	U5006	HYD-CLN-STD-02	rahul verma	400	1	poor service quality

Column A = booking_id, C = service_code, E = partner_name (notice the inconsistent spacing and capitalization — very normal in real data), G = price, H = rating, I = cancel_reason_text.

1. LEFT, MID & RIGHT

Real world reference:

Imagine a word written on a strip of paper, and you're told: "cut off the first 3 letters," or "grab 3 letters starting from the 5th one," or "cut off the last 2 letters." That's exactly LEFT, MID and RIGHT — three different ways to slice a piece off a longer piece of text, based on position.

In Urban Company's world:

The service_code "BLR-SAL-DLX-02" secretly contains four pieces of information squashed into one string: city (BLR), category (SAL), tier (DLX), and a sequence number (02). LEFT grabs the city from the start, MID grabs the category from the middle, and RIGHT grabs the sequence number from the end.

Syntax:

```
=LEFT(text, num_chars)          → characters from the START =MID(text, start_pos, num_chars) → characters from a MIDDLE position =RIGHT(text, num_chars)          → characters from the END  Example: =LEFT(C2,3)   =MID(C2,5,3)   =RIGHT(C2,2)
```

See it in the table:

	A	C	K	L	M
1	booking_id	service_code	City (LEFT)	Category (MID)	Seq (RIGHT)
2	U5001	BLR-SAL-DLX-02	=LEFT(C2,3) → BLR	=MID(C2,5,3) → SAL	=RIGHT(C2,2) → 02

Counting characters in "BLR-SAL-DLX-02": positions 1-3 are BLR, position 5 onward for 3 characters is SAL (position 4 is the hyphen), and the last 2 characters are 02.

2. LEN

Real world reference:

If you lay out a word letter by letter, LEN just counts how many letters are in the line — including spaces. It's a ruler for text.

In Urban Company's world:

Partner names in this data have a messy habit of picking up stray spaces during entry — " priya sharma " has extra spaces you can barely see. LEN reveals the problem: if the name LOOKS like 12 characters but LEN says 15, you know there are 3 hidden spaces sitting in there.

Syntax:

```
=LEN(text)  Example: =LEN(E2)
```

See it in the table:

	A	E	K
1	booking_id	partner_name	Character Count
2	U5001	" priya sharma "	=LEN(E2) = 15
6	U5005	"Priya Sharma"	=LEN(E6) = 12

Same person, same name — but U5001's version is 3 characters longer because of hidden leading/trailing spaces. LEN is often the first clue that a column needs cleaning.

3. FIND & SEARCH

Real world reference:

Imagine playing "Where's Wally" but for a specific word inside a sentence — you're not extracting anything, just pointing and saying "there, at word number 9." FIND and SEARCH both tell you the POSITION where a snippet of text starts inside a longer piece of text. The only difference: FIND is picky about capital vs lowercase letters; SEARCH doesn't care.

In Urban Company's world:

Support wants to flag every cancellation that mentions being "late." You need to find WHERE that word sits inside the free-text cancel_reason_text column — not extract it, just confirm it's there and where.

Syntax:

```
=FIND(search_text, text)     → case-SENSITIVE ("Late" ≠ "late") =SEARCH(search_text, text)   → case-INsensitive ("Late" = "late")  Example: =SEARCH("arrived", I2)
```

See it in the table:

	A	I	K
1	**booking_id**	**cancel_reason_text**	**Position of 'arrived'**
3	U5002	partner arrived late	=SEARCH("arrived",I3) = 9

"arrived" starts at the 9th character of the sentence. Using FIND("Arrived",I3) with a capital A would return a #VALUE! error here — SEARCH is the safer default when you're not sure of the exact capitalization.

4. SPLIT

Real world reference:

Imagine a Kit-Kat bar with grooves already built in — snap along the groove and it breaks into neat, equal pieces automatically. SPLIT does that to text: give it a delimiter (the "groove" character, like a hyphen), and it breaks one cell into several clean cells.

In Urban Company's world:

Instead of pulling city, category and tier out one at a time with LEFT/MID/RIGHT, SPLIT can do all of it in a single formula — breaking "BLR-SAL-DLX-02" apart at every hyphen, spilling the four pieces into four separate cells automatically.

Syntax:

```
=SPLIT(text, delimiter)  Example: =SPLIT(C2, "-")
```

See it in the table:

	A	C	K	L	M	N
1	**booking_id**	**service_code**	**City**	**Category**	**Tier**	**Seq**
2	U5001	BLR-SAL-DLX-02	BLR	SAL	DLX	02

=SPLIT(C2,"-") typed ONCE in cell K2 automatically fills K2, L2, M2 and N2 — Sheets "spills" the result across as many cells as it needs.

5. SUBSTITUTE

Real world reference:

Like using find-and-replace in a document — you tell it exactly what word to look for and what to swap it with, everywhere it appears.

In Urban Company's world:

You want a human-readable version of the service code for a support dashboard — turning "BLR-SAL-DLX-02" into "BLR SAL DLX 02" by replacing every hyphen with a space, without touching anything else in the string.

Syntax:

```
=SUBSTITUTE(text, old_text, new_text)  Example: =SUBSTITUTE(C2, "-", " ")
```

See it in the table:

	A	C	K
1	booking_id	service_code	Readable Code
2	U5001	BLR-SAL-DLX-02	=SUBSTITUTE(C2,"-"," ") → BLR SAL DLX 02

6. TRIM & CLEAN

Real world reference:

TRIM is like tidying a messy sentence that has too many spaces between words and around the edges — it squeezes everything down to single spaces and removes the edges entirely. CLEAN is a step further: it strips out weird invisible characters that sometimes sneak in when text is copy-pasted from other apps — the kind you can't even see, but that quietly break your formulas.

In Urban Company's world:

" priya sharma " and "Priya Sharma" are the same person, but Sheets treats them as different text because of the stray spaces — which means a COUNTIF for "Priya Sharma" would MISS the first one entirely. TRIM fixes that, cleaning every partner name down to a consistent, comparable format before you do any counting or matching.

Syntax:

```
=TRIM(text)    → removes extra/leading/trailing spaces =CLEAN(text)   → removes invisible non-printing characters  Example: =TRIM(E2)
```

See it in the table:

	A	E	K
1	booking_id	partner_name (raw)	TRIM(partner_name)
2	U5001	" priya sharma "	=TRIM(E2) → "priya sharma"
7	U5006	"rahul verma"	=TRIM(E7) → "rahul verma"

Notice U5006 had THREE spaces between "rahul" and "verma" — TRIM squeezes multiple internal spaces down to exactly one, not just the edges.

7. UPPER, LOWER & PROPER

Real world reference:

Think of a light switch for capitalization. UPPER shouts everything in CAPITALS. LOWER whispers everything in lowercase. PROPER dresses it up smartly — Capitalizing The First Letter Of Each Word, the way a name should look on an invoice.

In Urban Company's world:

Partner names arrive in every possible case — "RAHUL VERMA", "rahul verma" — and a support dashboard needs them looking consistent and professional. PROPER fixes the capitalization; combined with TRIM from the last concept, it fixes spacing too.

Syntax:

```
=UPPER(text)     =LOWER(text)     =PROPER(text)  Example: =PROPER(TRIM(E2))
```

See it in the table:

	A	E	K
1	**booking_id**	**partner_name (raw)**	**PROPER(TRIM(...))**
3	U5002	"RAHUL VERMA"	=PROPER(TRIM(E3)) → "Rahul Verma"
7	U5006	"rahul verma"	=PROPER(TRIM(E7)) → "Rahul Verma"

Both messy versions of "Rahul Verma" now produce the IDENTICAL clean output — which means they'll finally count as the same person in a COUNTIF or a pivot table.

8. VALUE

Real world reference:

Imagine a number written on a piece of paper versus the same number typed into a calculator — they look identical to your eye, but only one of them can actually be added or compared by a machine. VALUE takes a number that's secretly stored as TEXT and converts it into a real, usable number.

In Urban Company's world:

customer_pincode sometimes gets imported as text (often shows up left-aligned instead of right-aligned in the cell — the giveaway). You can't do numeric comparisons like "pincode > 560000" on text — VALUE converts it into a proper number first.

Syntax:

```
=VALUE(text)  Example: =VALUE(F2)
```

See it in the table:

	A	F	K
1	**booking_id**	**customer_pincode (text)**	**VALUE(pincode)**
2	U5001	"560001" (stored as text)	=VALUE(F2) → 560001 (a real number)

9. TEXTJOIN & CONCAT

Real world reference:

If SPLIT is snapping a Kit-Kat into pieces, TEXTJOIN and CONCAT are melting separate pieces of chocolate back into one bar. CONCAT glues text together with nothing in between. TEXTJOIN does the same, but lets you choose a separator (like a space, comma or dash) to glue with, and can join many pieces at once.

In Urban Company's world:

Support wants a single readable "search tag" for each booking, combining the category and the partner's name — something like "SAL | Priya Sharma" — built from pieces you've already extracted this session.

Syntax:

```
=CONCAT(text1, text2)                          → joins exactly two pieces, no separator
=TEXTJOIN(delimiter, ignore_empty, text1, text2, ...) → joins many pieces with a
chosen separator  Example: =TEXTJOIN(" | ", TRUE, MID(C2,5,3), PROPER(TRIM(E2)))
```

See it in the table:

	A	C	E	K
1	booking_id	service_code	partner_name	Search Tag
2	U5001	BLR-SAL-DLX-02	priya sharma	=TEXTJOIN(" | ",TRUE,MID(C2,5,3),PROPER(TRIM(E2))) → SAL | Priya Sharma

One formula, built entirely from pieces you already know how to make — MID for the category, TRIM+PROPER for a clean name, TEXTJOIN to glue them into one tag.

Coming to class

You now know how to pull structured information out of a messy code, clean up sloppy human-entered text, and rebuild new readable fields from the pieces. In class, we'll parse the full service_code column and finally test whether deluxe services in Bangalore really are the problem.